CSCE 883 midterm study guide

What to prioritize

The professor allows one page, single sided, and describes conceptual questions, simple Bayesian/MLE/tree/metric derivations and calculations, code understanding, neural networks, and backpropagation. Your classroom notes add a hand convolution and interpretation of a short diffusion noise-prediction loss. Practice explaining existing code; none of the practice questions requires writing code.

Highest priority: reproduce the non-AI-focused homework derivations without a solution in front of you: HW1 Q3; HW2 Q1–5 and Q7; HW3 Q1–3. Also calculate classification metrics by hand from HW2 Q8, although it originally required code. **Next:** explain KNN/scaling and the flawed HW1 pipeline, then interpret the AI-assisted cascade forest, DNA, and regression experiments. **Then:** review the architecture-specific ideas from lectures 6–9, especially CNN, RNN/LSTM, GAN, and diffusion.

The homework labels are evidence about study emphasis, not an exact rule about exam format. HW1 permits AI coding generally; HW2 Q8 explicitly asks for your own metric implementation. An unlabeled coding assignment can still become a code-reading or conceptual exam question.

Homework-to-exam map

Source	What you should be able to do	Most useful rehearsal
HW1 Q1	Euclidean KNN; majority vote; nearest-neighbor tie policy; train-only min–max/standard scaling; effect of K	Hand distances and votes; explain array axes and preprocessing
HW1 Q2	Explain your two selected KNN papers' problem, mechanism, evidence, limitation; propose a falsifiable improvement	Brief oral summaries; lower priority than calculations
HW1 Q3	Priors, categorical conditional counts, posterior scores, prediction, zero-count smoothing	Red/Domestic/SUV example, then a changed query
HW1 Q4	Feature/label separation, no-header loading, NB distribution assumptions, macro metrics, confusion matrix	Explain encoded categories versus continuous measurements
HW1 Q5 (explicit AI investigation)	Audit leakage, included labels, wrong distances, tie behavior; compare held-out models	Read `q5_starter_classifier.py` and explain each substantive defect
HW2 Q1	Discrete PMF with grouped probability mass; log likelihood; MLE of theta	Change category counts and derive theta again
HW2 Q2	Zero-location Laplace scale MLE	Derive mean absolute observation, not standard deviation
HW2 Q3	Gaussian mean and variance MLE	Show derivatives and explain N versus N-1
HW2 Q4	Poisson rate MLE	Derive sample mean from log likelihood
HW2 Q5	Gaussian probabilistic-regression NLL with learned mean and standard deviation	Explain squared residual/variance and log-standard-deviation terms
HW2 Q6 (AI coding allowed)	Cascade forests; augment inputs with class probabilities; compare with RF fairly	Explain out-of-fold features and validation-based stopping
HW2 Q7	Manually build tree using information gain, including impure/unsplittable leaves	Compute every weighted child entropy and recurse

Source	What you should be able to do	Most useful rehearsal
HW2 Q8	Class 0 as positive; TP/FP/TN/FN; accuracy/precision/recall; ROC/AUC; CV/ LOOCV	Read metric code; select correct probability column
HW3 Q1A	Sigmoid chain-rule gradient bound; why weight magnitudes matter; residual Jacobian	Derive (1/4)^L under the stated scalar weight bound
HW3 Q1B	Every CNN shape, per-filter and total parameters, flattening, dense parameters	Recalculate the original 3×24×24 architecture
HW3 Q2A/B	Four gradients for two hidden units; sigmoid and scaled Softplus; simultaneous updates	Derive from BCE and logits, then swap hidden activation
HW3 Q2C	Analytic versus autodiff versus central finite differences	Explain numerical disagreement and step-size tradeoff
HW3 Q3	Leakage and test-selection bias; correct experimental sequence; group split	Audit a short paragraph or code excerpt
HW3 Q4 (AI encouraged)	DNA frequency LR vs flattened MLP vs 1D CNN; one-hot channels; frozen shuffling control	Explain what composition-preserving perturbation establishes
HW3 Q5 (AI encouraged)	Mean/ridge/MLP baselines; MSE, scaling, capacity/regularization; original-unit regression metrics	Interpret curves, paired results, and negative R^2

1. Foundations and KNN — lectures 1–2

- ☐ Distinguish supervised classification, supervised regression, unsupervised learning, and reinforcement learning; representation learning versus hand-engineered features.
- ☐ Identify **model/hypothesis class**, **loss**, and **optimizer** as separate choices. Parameters are learned; hyperparameters such as K, depth, and learning rate are selected.
- ☐ Explain generalization, inductive bias, underfitting, overfitting, and why training performance alone is insufficient. A more expressive model need not generalize better.
- ☐ Describe a version space as hypotheses consistent with observed examples; margin as separation from a boundary.
- ☐ Explain KNN as a lazy learner: store examples, perform distance search at prediction time; small K is noise-sensitive, large K can blur boundaries. Irrelevant dimensions and disparate scales degrade meaningful neighbor comparisons.

For vectors, $d(x, q) = \sqrt{\sum_j (x_j - q_j)^2}$. Squared distance gives the same neighbor ordering. A signed sum $\sum_j (x_j - q_j)$ is not Euclidean distance: signs cancel and it can be negative.

Min–max: $x'_j = (x_j - \min_{train,j})/(\max_{train,j} - \min_{train,j})$. Standardization: $x'_j = (x_j - \mu_{train,j})/s_{train,j}$. Fit on training **features**, then reuse the transformation. Handle constant columns explicitly. Test points can legitimately fall outside [0,1] after training-based min–max scaling.

HW1 tie rule: when the top class vote is tied, use the class of the nearest neighbor among tied classes. `bincount(...).argmax()` instead selects the smallest class index on a tie. Distance-weighted voting gives closer neighbors more influence; a zero distance needs explicit treatment. KNN regression averages neighbors' targets.

2. Bayes, Naive Bayes, and decisions — lectures 2–4

- ☐ Name prior, likelihood, evidence, and posterior; distinguish $P(x \mid c)$ from $P(c \mid x)$.
- ☐ Compute a categorical NB prediction by counts, with the correct class-specific denominators.
- ☐ State **conditional** independence given the class; it does not assert unconditional independence.
- ☐ Explain categorical, Bernoulli, multinomial, and Gaussian feature models. Integer category codes do not automatically justify Gaussian assumptions or numeric distance.
- ☐ Explain minimum expected risk when error costs differ, and the difference between MLE, MAP, and full Bayesian prediction.

$$P(c \mid x) = \frac{P(x \mid c)P(c)}{\sum_{c'} P(x \mid c')P(c')}, \qquad s_c = P(c)\prod_j P(x_j \mid c), \qquad \hat{c} = \arg\max_c s_c.$$

Normalize scores only if a posterior probability is requested. In code, log scores avoid underflow: $\log P(c) + \sum_j \log P(x_j \mid c)$. Laplace smoothing for a feature with V possible values is $(n_{j,v,c}+1)/(n_c+V)$; smooth each feature's conditional distribution separately.

Original HW1 check: Red/Domestic/SUV has Yes prior 5/10 and conditionals 3/5, 2/5, 1/5; No has prior 5/10 and conditionals 2/5, 3/5, 3/5. Scores are 0.024 and 0.072, so predict No; normalized Yes posterior is 0.25.

Expected risk: $R(a \mid x) = \sum_c \lambda_{a,c} P(c \mid x)$. With zero correct-class cost, false-positive cost C_{FP} and false-negative cost C_{FN}, choose positive when $P(+ \mid x) > C_{FP}/(C_{FP}+C_{FN})$. Under 0/1 loss this reduces to the most probable class. With a constant rejection cost r, compare r against the smallest classification risk.

MLE maximizes $p(D \mid \theta)$; MAP maximizes $p(D \mid \theta)p(\theta)$; posterior predictive inference integrates $p(x_{new} \mid \theta)$ over $p(\theta \mid D)$. For squared-error parameter estimation, the posterior mean is the Bayes estimator. A uniform prior can make MAP and MLE coincide.

3. MLE and regression losses — lecture 4; HW2 Q1–5

Always start with $L(\theta) = \prod_i p(x_i \mid \theta)$ under independence, then $\ell = \log L = \sum_i \log p(x_i \mid \theta)$. Differentiate, solve under the parameter constraints, and check a maximum and boundaries. Logs preserve the maximizing parameter. Remove only terms independent of **all parameters being optimized**.

Model	Log likelihood up to constants	MLE
Bernoulli p	$S\log p + (N-S)\log(1-p)$	$\hat{p} = S/N$
HW2 Q1 PMF: $(2\theta/3, \theta/3, 2(1-\theta)/3, (1-\theta)/3)$	$(n_0+n_1)\log\theta + (n_2+n_3)\log(1-\theta)$	$\hat{\theta} = (n_0+n_1)/N$
Laplace, fixed location 0, scale b>0	$-N\log(2b) - \sum_i \lvert x_i\rvert/b$	$\hat{b} = \sum_i \lvert x_i\rvert/N$ if sum>0
Poisson lambda	$(\sum_i x_i)\log\lambda - N\lambda$	$\hat{\lambda} = \bar{x}$
Gaussian mean mu, variance v>0	$-N\log v/2 - \sum_i (x_i-\mu)^2/(2v)$	$\hat{\mu} = \bar{x}$, $\hat{v} = \sum_i (x_i-\bar{x})^2/N$
Multinomial probabilities	$\sum_k n_k \log p_k$, $\sum_k p_k = 1$	$\hat{p}_k = n_k/N$

Gaussian derivative steps: $\partial\ell/\partial\mu = \sum_i (x_i-\mu)/v = 0$; $\partial\ell/\partial v = -N/(2v) + \sum_i (x_i-\mu)^2/(2v^2) = 0$. The MLE variance uses N; N-1 gives the usual unbiased sample variance. Report standard deviation as the

square root if asked for sigma. For all-zero Laplace observations, likelihood is unbounded as b approaches zero; there is no positive-scale maximizer. For identical Gaussian observations, a similar zero-variance degeneracy occurs. Bernoulli/grouped-PMF endpoints and Poisson lambda=0 may be valid boundary answers.

Probabilistic neural regression: $y_i \mid x_i \sim \mathcal{N}(\mu_\theta(x_i), \sigma_\theta(x_i)^2)$ with sigma>0.

$$\text{NLL} = \sum_i \left[\tfrac{1}{2}\log(2\pi) + \log\sigma_i + \frac{(y_i - \mu_i)^2}{2\sigma_i^2} \right].$$

For a network outputting $s_i = \log\sigma_i^2$, the variable part is $\frac{1}{2}[s_i + (y_i - \mu_i)^2 e^{-s_i}]$. A positive output can also use Softplus plus a small floor. With fixed variance, Gaussian MLE reduces to minimizing squared error. With learned variance, **retain the log-variance term**: it prevents arbitrarily large variance from making the residual term free. Laplace observation noise yields absolute-error loss.

Linear regression: $\hat{y} = w^T x + b$. Ridge minimizes squared error plus $\lambda\|w\|_2^2$ (conventions for averaging change the scale of lambda). Logistic regression: $p = \sigma(w^T x + b)$ and binary cross-entropy. Logistic regression is a classifier despite its name. Equal class covariances in Gaussian discriminant models produce linear boundaries; class-specific covariances generally produce quadratic boundaries. Mahalanobis squared distance is $(x-\mu)^T\Sigma^{-1}(x-\mu)$.

Estimator MSE decomposes into bias^2 + variance; prediction squared error also contains irreducible noise under the standard regression assumptions. Increasing capacity can lower training error while increasing estimation variability. Regularization and validation help choose the tradeoff.

4. Metrics and reliable evaluation — lecture 3; HW2 Q8; HW3 Q3–5

First declare the positive class and confusion-matrix axes. For rows=true and columns=predicted in order [0,1], class-1-positive counts are $\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$. If class 0 is positive, relabel the roles before using formulas.

$$Accuracy = \frac{TP+TN}{N}, \quad Precision = \frac{TP}{TP+FP}, \quad Recall = TPR = \frac{TP}{TP+FN},$$
$$Specificity = \frac{TN}{TN+FP}, \quad FPR = \frac{FP}{FP+TN}, \quad F1 = \frac{2TP}{2TP+FP+FN}.$$

- ☐ Compute each class's metrics one-versus-rest. Macro averages give classes equal weight; weighted averages use class support; micro averages pool counts. Macro-F1 is the average of class F1s, not F1 calculated from macro precision and recall.
- ☐ Explain why a majority-only classifier can have high accuracy and zero minority recall. Binary balanced accuracy is the mean of the two class recalls.
- ☐ Trace threshold changes: lowering a positive-score threshold weakly increases TPR and FPR. ROC plots TPR against FPR across thresholds; AUC summarizes ranking, not accuracy at one threshold. AUC=0.5 is chance-level ranking; use probability/decision scores, not hard labels.
- ☐ Interpret AUC as the probability a random positive outranks a random negative, with half credit for ties. Match the probability column to the declared positive class using the model's class ordering.
- ☐ Explain k-fold CV and LOOCV (N folds, train on N-1, evaluate on one); fit preprocessing anew inside each fold. Hyperparameter tuning requires a validation layer or nested CV if CV performance itself is the final estimate.

Regression: $MAE = N^{-1}\sum|y_i - \hat{y}_i|$, $MSE = N^{-1}\sum(y_i - \hat{y}_i)^2$, $RMSE = \sqrt{MSE}$, $R^2 = 1 - \sum(y_i - \hat{y}_i)^2 / \sum(y_i - \bar{y}_{test})^2$. Negative R^2 means worse squared error than the evaluated set's own mean benchmark. A deployable mean baseline uses the **training** mean; it is not necessarily the denominator's test mean.

MAE and RMSE have original target units; invert target scaling before scoring. R^2 is undefined under its direct formula when the evaluated targets are constant.

Correct workflow: establish test split → split development into train/validation → fit preprocessing on training → train/tune using validation → choose epoch and restore its checkpoint → freeze choices → evaluate test. Splitting after scaling leaks information. Repeatedly selecting the best test model biases the estimate; repetition does not repair it. Group splits by subject/material/sequence family are needed to evaluate unseen groups; random row splits can share related entities across partitions.

HW3 comparison rules worth understanding: shared splits and paired model differences; mean ± **sample** standard deviation across seeds; overlapping split results are not independent and their SD is not a confidence interval. Training curves diagnose fitting/optimization; validation curves guide selection; post-test subgroup/residual analysis is descriptive, not a new tuning set.

5. Decision trees, RF, and cascade forests — lecture 5; HW2 Q6–7

$$H(S) = -\sum_c p_c \log_2 p_c, \quad H(S \mid A) = \sum_v \frac{|S_v|}{|S|} H(S_v), \quad IG(S, A) = H(S) - H(S \mid A).$$

Use $0 \log 0 = 0$. Pure binary nodes have entropy 0; a 50/50 node has entropy 1 bit. Choose the largest information gain, then **recompute using only each child subset**. Stop for purity or no useful remaining split; label unresolved leaves by majority with a declared tie policy. Conflicting labels for identical feature vectors cannot be perfectly separated by those features.

HW2 Q7's original rows are $(x_1, x_2, y) = (0,1,0), (1,1,0), (0,0,1), (1,0,0), (0,1,0), (0,1,1)$. Root entropy is $H(2/6) = 0.9183$. Gain for x_1 is 0.2516 and for x_2 is 0.0441, so split x_1 first. Its $x_1 = 1$ branch is pure class 0; on $x_1 = 0$, $x_2 = 0$ is class 1 and $x_2 = 1$ has two zeros and one one, so majority class 0. The last leaf cannot be made pure with the two available features.

- ☐ Explain greedy ID3; numerical threshold candidates between adjacent distinct sorted values; pruning to limit overfitting.
- ☐ Know Gini $= 1 - \sum_c p_c^2$ and weighted child impurity. Gain ratio is IG divided by split entropy $-\sum_v (|S_v|/|S|) \log_2 (|S_v|/|S|)$ when nonzero. Information gain can favor many-valued identifiers.
- ☐ Explain RF bootstrap sampling and a random subset of candidate features at **each split**; classification voting/probability aggregation versus regression averaging. Randomness reduces correlation among trees. Out-of-bag evaluation uses only trees whose bootstrap samples omit the evaluated example.
- ☐ Explain cascade forest layers: concatenate class-probability features from several forests with original features; later layers refine predictions. Training probability features should come from out-of-fold predictions so an example does not get its own fitted prediction as an apparently generalizable feature. Held-out inputs use predictors fitted on allowed training data; depth is selected with validation, not test scores.

6. Neural nets and backpropagation — lectures 6–7; HW3 Q1–2

Forward layer: $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$, $a^{(l)} = \phi(z^{(l)})$. Nonlinearities make stacked layers more expressive than a single affine map; a stack of affine layers alone collapses to one affine layer. Universal approximation is an existence result, not a guarantee of successful optimization or generalization. Dense layer parameters: $(n_{in} + 1)n_{out}$ with biases.

$$\sigma(u) = \frac{1}{1 + e^{-u}}, \quad \sigma'(u) = \sigma(u)(1 - \sigma(u)) \leq \tfrac{1}{4}.$$

$$ReLU(u) = \max(0, u), \quad ReLU'(u) = 1 \ (u > 0), \ 0 \ (u < 0).$$

$$\phi_k(u) = \frac{\log(1 + e^{2ku})}{2k}, \quad \phi_k'(u) = \sigma(2ku).$$

ReLU uses a chosen subgradient at zero (commonly 0); smoothness everywhere is not required. ReLU can have dead units; it does not eliminate every gradient problem. Scaled Softplus approaches ReLU as k increases; its derivative is 1/2 at zero for finite k. Tanh derivative is $1 - \tanh^2 u$.

Match loss and output. Binary classification: one logit + BCE; mutually exclusive multiclass: logits + softmax cross-entropy; unconstrained regression: linear output + MSE. $softmax(z)_k = e^{z_k} / \sum_j e^{z_j}$; subtract the largest logit for stability. BCE from logit z is $\max(z, 0) - yz + \log(1 + e^{-|z|})$. Do not feed already-sigmoided outputs to a loss expecting logits.

For $h_1 = \phi(w_1 x), h_2 = \phi(w_2 x), z = w_3 h_1 + w_4 h_2, p = \sigma(z)$ with BCE:

$$\delta = \frac{\partial L}{\partial z} = p - y, \quad \frac{\partial L}{\partial w_3} = \delta h_1, \quad \frac{\partial L}{\partial w_4} = \delta h_2,$$

$$\frac{\partial L}{\partial w_1} = \delta w_3 \phi'(w_1 x) x, \quad \frac{\partial L}{\partial w_2} = \delta w_4 \phi'(w_2 x) x.$$

Derive the cancellation: $\partial L/\partial p = -y/p + (1-y)/(1-p)$ multiplied by $\partial p/\partial z = p(1-p)$ gives p-y. For half squared error with sigmoid output, the output delta instead is $(p - y)p(1 - p)$. Lecture 6's squared-error backprop and HW3's BCE therefore have different output factors.

Updates are $w_i^{new} = w_i^{old} - \eta \partial L/\partial w_i$, with every gradient evaluated at the old weights. General hidden delta: $\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot \phi'(z^{(l)})$; $\partial L/\partial W^{(l)} = \delta^{(l)} (a^{(l-1)})^T$, and bias gradient is delta (sum/average over examples as appropriate).

For scalar $h_l = \sigma(w_l h_{l-1})$, $|\partial h_L/\partial x| = |\prod_l w_l \sigma'(w_l h_{l-1})| \leq (1/4)^L$ **when** $|w_l| \leq 1$. Without that bound, weights/Jacobians may amplify gradients. For vector residual $h_{l+1} = h_l + F_l(h_l)$, $J = I + J_{F_l}$ provides a direct gradient path but does not guarantee stability.

Central difference: $\partial L/\partial w_i \approx [L(w + \epsilon e_i) - L(w - \epsilon e_i)]/(2\epsilon)$. Compare against analytic and automatic gradients; float64 helps. Too-large epsilon creates truncation error, too-small epsilon magnifies cancellation/ roundoff. Nondifferentiable points complicate such checks.

Optimization and regularization: SGD uses batches; momentum accumulates a direction; Adagrad/ RMSProp adapt learning rates via squared-gradient information; Adam combines moving first/second moments. Learning rate controls step size. Dropout removes random activations during training and is disabled for evaluation; inverted dropout scales surviving training activations by $1/(1-p)$ (older slide conventions scale at inference instead). Weight decay discourages large weights; early stopping uses validation loss. Depth can improve representation efficiency while making optimization harder.

7. CNNs — lecture 7; HW3 Q1B/Q4; your hand-calculation warning

- ☐ Explain local receptive fields, shared kernels, channels, feature maps, stride, padding, pooling, and flattening.
- ☐ Calculate every output cell, adding bias once per cell and summing **across input channels**.
- ☐ Track output dimensions independently from parameter count. Increasing image size changes activations, not a fixed convolution's parameter count.
- ☐ Distinguish translation equivariance of convolution from approximate shift tolerance introduced by pooling. Boundary effects and strides matter; CNNs are not universally translation invariant.

For one spatial dimension with kernel K, stride S, padding P, dilation D:

$$N_{out} = \left\lfloor \frac{N_{in} + 2P - D(K-1) - 1}{S} \right\rfloor + 1.$$

Conv2D parameters with groups 1: $C_{out}(C_{in}K_HK_W + 1)$. Each output filter spans **all** input channels. Conv1D: $C_{out}(C_{in}K + 1)$. Pooling has no learned parameters. Flattening preserves all elements, CHW.

Hand operation under the usual NN cross-correlation convention:

$$Y_{o,i,j} = b_o + \sum_c \sum_u \sum_v K_{o,c,u,v} X_{c,iS+u-P,jS+v-P}.$$

The kernel is not flipped. Mathematical convolution flips it; state the convention before calculating. Practice a valid stride-1 2×2 kernel over a 3×3 image, then add a second channel, padding, or a pooling step.

Original HW3 dimensions: $3 \times 24 \times 24 \to 12 \times 22 \times 22 \to 12 \times 11 \times 11 \to 128 \times 9 \times 9 \to 128 \times 4 \times 4$. Conv1 has 27 weights + 1 bias per filter, total 336; Conv2 has 108 weights + 1 bias per filter, total 13,952. Flatten length 2,048; dense 10 outputs adds 20,490 parameters. Pools add 0.

DNA CNN input is four nucleotide channels × 250 sequence positions; kernel moves along positions. The MLP flattens to 1,000 features. Frequency features discard order; one-hot sequence representations retain it. Frozen within-sequence shuffling keeps composition and labels fixed while destroying many order cues. A performance drop supports sensitivity to order under this perturbation but can reflect distribution shift; it does not identify biological motifs. A frequency-only model's predictions should remain unchanged. Retraining on shuffled train/validation data answers a different question about information learnable after order is removed.

8. Other NN types — lectures 6–9

Type	Necessary idea and compact equation	What to explain
MLP/DNN	$a_l = \phi(W_l a_{l-1} + b_l)$	Nonlinear feature composition; task-appropriate head and loss; capacity vs generalization
Residual net	$h_{l+1} = h_l + F_l(h_l)$	Identity path in backprop; shape compatibility for addition
RNN	$h_t = \phi(W_x x_t + W_h h_{t-1} + b)$	Shared weights over time; state/ memory; unroll and backpropagate through time; exploding/vanishing gradients
LSTM	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, $h_t = o_t \odot \tanh(c_t)$	Forget/input/output sigmoid gates regulate retention, writing, and exposure; additive cell path supports long dependencies
GRU (briefly in lecture 7)	Gated mixture of old state and candidate state	Reset/update gates; simpler than LSTM, no separate cell state; lower priority than LSTM
Autoencoder	$z = E(x)$, $\hat{x} = D(z)$, minimize reconstruction loss	Bottleneck/representation learning; target is input; denoising AE predicts clean input from corrupted input

Type	Necessary idea and compact equation	What to explain
Hopfield net (lecture 6)	Recurrent, symmetric weights; energy-based associative memory	Recall stored patterns from incomplete/ corrupted cues; lower priority than homework-linked models
GAN/DCGAN	Noise $z \to G(z)$; discriminator distinguishes real/fake	Two competing learned models, alternating updates, instability/mode collapse; DCGAN uses convolutional architectures
Diffusion/U-Net	Fixed forward corruption; learned timestep-conditioned reverse denoising	Noise target, schedule, iterative generation, encoder/decoder skip connections; latent-space cost reduction

For LSTM, the usual gates are $f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$, similarly i and o, with $\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c)$. Gate near 1 passes information; near 0 suppresses it. The direct cell-state path has factor f_t; full derivatives also include gate dependencies. LSTMs help with vanishing gradients, not guarantee their absence. Bidirectional RNNs use past and future sequence context and cannot directly serve a strictly causal online predictor.

Lecture 7 also includes encoder–decoder/sequence-to-sequence models, older attention examples, embeddings, unsupervised pretraining, and reinforcement-learning examples. Recognize their purpose and diagram; give them less time than explicit homework topics. Earlier recurrent attention may be in scope; **Lecture 10's transformer treatment is excluded**.

GAN formulas and code interpretation

$$\min_G \max_D V = \mathbb{E}_{x \sim p_{data}} \log D(x) + \mathbb{E}_z \log(1 - D(G(z))).$$

When minimizing a discriminator loss, negate both terms. A common non-saturating generator loss is $L_G = -\mathbb{E}_z \log D(G(z))$: fake samples are given a real target for the generator update. If D strongly rejects fake samples, the original saturating generator objective can have weak gradients. Mode collapse means poor diversity despite plausible individual samples. At the ideal matched-distribution equilibrium, D outputs 1/2. Implicit sampling does not provide an easy explicit normalized density. An adversarial objective does not guarantee absence of memorization/overfitting.

For discriminator training, detach generated samples so G is not updated. For generator training, gradients must pass through D into G, even if D's parameters are frozen for that step. Freezing parameters differs from placing the whole forward pass in a no-gradient context.

Diffusion formulas and the short loss example

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} x_{t-1}, \beta_t I), \qquad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

$$L_{simple} = \mathbb{E}_{x_0, t, \epsilon} \|\epsilon - \epsilon_\theta(x_t, t)\|^2.$$

This samples a chosen noisy timestep directly during training; it does not run every forward step. The schedule is fixed, and the network learns to estimate sampled noise conditional on noisy input and timestep. A vanilla epsilon-prediction model minimizes **noise MSE**, not necessarily clean-image or label error. Some

slide derivation pages use a reverse-mean/previous-step target; identify the parameterization from the actual code. Your classroom example specifically uses predicted-noise MSE.

```
noise = torch.randn_like(x0)
xt = abar[t].sqrt() * x0 + (1 - abar[t]).sqrt() * noise
pred_noise = model(xt, t)
return ((pred_noise - noise) ** 2).mean()
```

Read line by line: Gaussian target noise → closed-form corrupted sample → timestep-conditioned prediction → average squared residual over all tensor entries. Assume schedule coefficients are reshaped for batch/channel broadcasting. Gradient with respect to each predicted element is $2(pred - noise)/M$ for M elements. A timestep-dependent schedule matters because corruption strength changes; training loss alone does not perform generation.

Clean estimate from predicted noise: $\hat{x}_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t))/\sqrt{\bar{\alpha}_t}$. A common DDPM reverse mean is $\mu_\theta = \alpha_t^{-1/2}[x_t - \beta_t\epsilon_\theta(x_t, t)/\sqrt{1-\bar{\alpha}_t}]$. Sampling starts from Gaussian noise and repeatedly takes reverse steps, often adding prescribed reverse-step noise except at the final step. U-Net combines downsampling, upsampling, and matching-scale skip connections. Latent diffusion does corruption/denoising in an encoder's smaller representation and decodes to images. Conditioning guides what is generated; it is not the same as the timestep input.

Code-reading checklist

For any short method, explain (1) input and output shapes, (2) what each operation computes, (3) prediction and target, (4) loss reduction and gradient direction, (5) training versus inference, and (6) any evaluation defect.

- ☐ `[:, :-1]` selects features; `[:, -1]` selects labels. Scaling all columns includes labels.
- ☐ Squaring and summing feature differences must use the feature axis; sorting retrieves neighbor indices, which must also index the aligned labels.
- ☐ `fit_transform` learns preprocessing; `transform` reuses it. Test fitting and whole-dataset fitting leak information.
- ☐ Probability columns follow class ordering; class 0 is not automatically the second column.
- ☐ `.mean()` versus `.sum()` changes loss scale; shared broadcasting can silently introduce unintended residual dimensions.
- ☐ `zero_grad` → `forward` → `loss` → `backward` → `optimizer.step` clears accumulated gradients, computes prediction/objective, differentiates, then updates. `backward` alone does not update weights.
- ☐ Evaluation mode disables dropout; no-gradient mode prevents gradient tracking. These are separate controls.
- ☐ Validation chooses the model/epoch; test estimates the frozen procedure's performance. Seeds aid reproducibility, not unbiasedness by themselves.

How to use your one-page cheatsheet

Allocate the most space to operations you must reproduce: Bayes/NB score and smoothing; MLE likelihood→log→derivative template and four distribution results; Gaussian regression NLL; entropy/gain; metric definitions with both class conventions; CNN shape/parameter formulas; BCE and squared-error output deltas; four HW3 weight gradients; sigmoid/ReLU/Softplus derivatives; residual Jacobian; diffusion noising/loss equations. Add tiny reminders for GAN loss signs, LSTM gates, train-only preprocessing, and validation selection. Put worked steps rather than long prose on the sheet. Check the professor's rules on font size, handwriting, and calculators separately if they were specified elsewhere.

Study sequence: redo HW1 Q3 → HW2 Q1–5/Q7/Q8 → HW3 Q1–3 → trace AI-assisted pipeline/ experiment snippets → compare NN families → attempt the practice midterm → use the answer key to identify missing chain-rule factors or interpretation gaps.

Source references

Original lecture folder: `C:\Users\tbelcher\OneDrive-SCGSSM\Belcher\USC\CSPhD\Courses\CS883\SlideDecks`.

- `Lec1-intro.pptx`: slides 7–9, representation learning and deep-learning motivation.
- `Lec2-KNN-NaivBayes.pptx`: slides 2–11 foundations; 12–26 KNN; 27 onward Bayesian classification.
- `Lec3-Bayes-Evaluation.pptx`: slides 3–10 Bayes/risk; 12–13 generalization; 14 onward evaluation/ metrics.
- `Lec4-MLE-Regression.pptx`: slides 3–13 MLE/Bayes; 19–32 regression/bias/regularization; 33–46 multivariate models and NB.
- `Lec5-DT-RF.pptx`: slides 2–22 discriminants/logistic/softmax; 26–58 trees; 59–64 RF.
- `Lec6-ANNclassic.pptx` and `Lec6-ANNclassic(1).pptx`: feedforward/recurrent basics and backprop; see slides 30–39 for the squared-error derivation.
- `Lec7_DNN_HYLee.pdf`: sections "Tips for Deep Learning," "CNN," "RNN," "LSTM," and "Auto-encoder"; the homework CNN resembles the deck's architecture.
- `lec8_gan.pdf`: architecture, minimax objective, vanishing gradients, DCGAN, non-convergence, and mode collapse.
- `Lec9-diffusion.pdf`: forward process, reparameterization, reverse denoising, noise-predicting U-Net, latent space, and conditioning. The page labeled reparameterization has ambiguous prose; the direct $x_t \mid x_0$ formula describes forward corruption.
- Professor homework: HW1, HW2, HW3. HW3 pp. 2–3 are the main hand-derivation source; pp. 4–7 are the AI-assisted experimental tasks. HW2 p. 1 contains the embedded distribution formulas; p. 3 contains the Q7 dataset.

Existing student solutions were consulted as secondary context, especially to locate Homework 2's image-only questions; the professor's assignments and slides define the topic coverage. Class discussions beyond the details you supplied cannot be reconstructed from these files.